SIMATS ENGINEERING
ASSESSMENT TOOL 3

COURSE NAME: Artificial Intelligence **COURSE CODE: CSA1741**

Register Number	**192325130**
Name	**K. Sirisha**

COURSE OUTCOME COVERED

CO1: Apply AI basics such as intelligent agents and uninformed search methods to solve problems effectively.*(BL3)*

Assessment Tool Weightage: 10%

QUESTIONS:2 **Total Marks:20**

Descriptive Test

1. A smart home automation system is being designed to manage lighting, temperature, and security based on user behaviour and environmental conditions. The system must respond to real-time inputs such as motion detection, temperature changes, and time of day while also learning user preferences over time. In this context, explain the different types of intelligent agents (simple reflex, model-based, goal-based, and utility-based agents) and analyze how each type would function within this system. Justify which type of agent or hybrid approach would be most effective for ensuring comfort, energy efficiency, and security.

2. A robot is placed in an unknown maze and must find a path to the exit without prior knowledge of the layout. Explain how uninformed search strategies like Uniform Cost Search (UCS) and Iterative Deepening Search (IDS) can help solve this problem. Discuss the advantages and disadvantages of these algorithms in terms of time and space complexity. Relate the problem to different types of intelligent agents and explain how agent design affects decision-making. Suggest which combination of agent type and search strategy would be most effective and justify your choice.

Code of Conduct:

I ___K. Sirisha (192325130)___ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: K. Sirisha

RUBRICS FOR CALCULATION:

Criterion	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)	Max Marks
Criteria	**Level 4 (Exemplary)**	**Level 3 (Proficient)**	**Level 2 (Developing)**	**Level 1 (Beginning)**	**Max Marks**
Understanding of Concepts	Demonstrates thorough, accurate understanding of concepts	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor understanding; major misconceptions	5
Explanation	Detailed, well-explained answers with relevant examples	Clear explanation with adequate details	Limited explanation; lacks depth	Poor or unclear explanation	5
Application	Effectively applies concepts with strong analytical ability	Applies concepts with minor errors	Limited application with weak analysis	No application or incorrect analysis	4
Organization	Well-structured answers with logical flow and coherence	Organized with minor issues in flow	Basic structure, lacks clarity	Poor organization, incoherent answers	3
Accuracy	Completely accurate and covers all required points	Mostly accurate with minor omissions	Partially correct with missing points	Incorrect answers with major gaps	2
Clarity	Clear, neat, professional presentation	Understandable with minor issues	Limited clarity, readability issues	Poorly written, difficult to understand	1

Question 1: Intelligent Agents in a Smart Home Automation System

1. Introduction

An intelligent agent is anything that senses its environment through sensors and acts on it through actuators to reach a goal. A smart home system is a good example of an agent.

Part of the agent	In the smart home
Environment	The house, the people living in it, the weather and the time of day
Sensors (percepts)	Motion sensor, temperature sensor, light sensor, door/window sensor, clock
Actuators (actions)	Lights, fan / AC / heater, door locks, camera, alarm

The home is only partly observable (the system cannot see every room all the time) and it keeps changing. So the type of agent must be chosen carefully.

2. Types of Intelligent Agents and How Each Works in the Smart Home

2.1 Simple Reflex Agent

Meaning: It chooses an action using only the **current percept** and fixed **IF–THEN rules**. It has no memory and no learning.

In the smart home:

- IF motion is detected AND it is night THEN switch on the hallway light.
- IF room temperature is above 30 °C THEN switch on the AC.
- IF a door opens AND the alarm mode is ON THEN sound the alarm.

Strength: Very fast, simple and cheap. Good for urgent actions.

Weakness: It forgets everything. If a person sits still and reads, the motion sensor shows “no motion” and the light is switched off. It cannot learn the user’s likes and cannot handle a situation that is not in its rules.

2.2 Model-Based Reflex Agent

Meaning: It keeps an **internal model (memory) of the world**. It remembers things it cannot see right now and knows how the world changes and how its own actions change it.

In the smart home:

- It keeps “room occupied = YES” for 15 minutes after the last motion, so the light stays on for a person who is sitting still.
- It remembers that the front door is locked and that a window was left open.
- It notices that the temperature has been rising for one hour and starts the AC early.

Strength: Works well when the environment is only partly visible. Makes fewer wrong decisions than a simple reflex agent.

Weakness: It still follows fixed rules. It has no goal, so it cannot plan ahead or pick the best choice among many.

2.3 Goal-Based Agent

Meaning: It has a **goal** (a situation it wants to reach). It thinks ahead ("what will happen if I do this?") and uses search and planning to pick actions that lead to the goal.

In the smart home:

- Goal: "House should be 24 °C at 6 pm when the owner returns." The agent plans the time to start the AC.
- Goal: "House must be secure by 10 pm." The agent plans the steps: close the windows, lock the doors, switch on the cameras and switch off unused lights. If a window is open, it changes its plan.

Strength: Flexible. Changing the goal changes the behaviour, so there is no need to rewrite all the rules.

Weakness: It only knows whether the goal is reached or not. When there are many ways to reach the goal, it cannot say which way is better (cheaper or more comfortable).

2.4 Utility-Based Agent

Meaning: It uses a **utility function**, which gives a number showing how good or "happy" a situation is. It picks the action with the **highest expected utility**, so it can balance goals that conflict with each other.

In the smart home:

- AC at 22 °C gives high comfort but a high electricity bill. AC at 26 °C saves power but is less comfortable. The agent adds up comfort, energy saving and security and chooses a balanced value such as 24–25 °C.
- When nobody is at home, the value of saving energy becomes higher, so the AC is switched off.
- It learns over time. If the user keeps setting 25 °C at night, the utility function is updated to prefer 25 °C.

Strength: Best for balancing comfort, cost and safety. It can handle uncertainty and learn user behaviour.

Weakness: Harder to design and needs more data and computing power.

3. Summary Comparison

Point	**Simple reflex**	**Model-based**	**Goal-based**	**Utility-based**
Decides using	Current percept only	Percept + memory of the world	Goal + future prediction	Utility value (how good)
Memory	No	Yes	Yes	Yes
Plans ahead	No	No	Yes	Yes
Smart home example	Motion → light ON	Remembers room is occupied	Warm house by 6 pm	Best balance of comfort and cost
Speed	Fastest	Fast	Slower	Slowest

4. Analysis: Comfort, Energy Efficiency and Security

Requirement	What the system needs	Most helpful agent	Reason
Comfort	Learn user preferences and prepare the home before the user needs it	Utility-based + Goal-based	Utility learns the preferred settings; goals plan the timing (e.g., warm room by 6 pm).
Energy efficiency	Know who is in which room and compare the cost of different choices	Model-based + Utility-based	Model tracks occupancy; utility balances comfort against power use.
Security	React instantly to danger and remember the state of doors and windows	Simple reflex + Model-based	Reflex rules act at once; the model remembers which doors are locked.

5. Best Choice: A Hybrid (Layered) Agent

No single agent type is enough, so the **best approach is a hybrid agent** that joins all four types in layers.

Layer	Agent type	Job in the smart home
Layer 1 – Emergency	Simple reflex	Instant reaction: sound the alarm for break-in, smoke or gas leak.
Layer 2 – Awareness	Model-based	Keeps track of who is home, which rooms are used, door/window status and temperature trend.
Layer 3 – Planning	Goal-based	Plans actions to meet goals such as “warm home by 6 pm” and “secure by 10 pm”.
Layer 4 – Decision and learning	Utility-based (with learning)	Balances comfort, energy and security and learns the user’s habits over time.

Justification:

- A simple reflex agent alone is too basic. It cannot remember or learn.
- A utility-based agent alone is too slow for emergencies, because it takes time to calculate while an intruder may already be entering.
- The hybrid agent is **fast when the situation is urgent** and **smart when there is time to think**.

Example: The owner returns home at 7 pm. The model-based layer knows the house was empty all day and the room is hot. The goal-based layer started the AC at 6:30 pm to reach the target. The utility-based layer chose 25 °C (the owner’s usual setting) to save power. At night, if a window sensor shows a break-in, the reflex layer sounds the alarm immediately.

6. Conclusion

Each agent type has a useful role: simple reflex for speed, model-based for memory, goal-based for planning and utility-based for balanced decisions. A **hybrid agent** combines all of them and gives the best **comfort, energy efficiency and security** for a smart home.

Question 2: Robot in an Unknown Maze – Uninformed Search and Agent Design

1. Introduction

A robot is placed in an unknown maze and must find the exit. It has **no map and no extra hint** (such as the distance to the exit). Search methods that use only the problem definition, and no hints, are called **uninformed (blind) search strategies**. Two useful ones are **Uniform Cost Search (UCS)** and **Iterative Deepening Search (IDS)**.

2. Problem Formulation

Element	In the maze problem
Initial state	The cell where the robot starts
Actions	Move forward, turn left, turn right, go back
Transition	The new cell the robot reaches after an action
Goal test	Is the robot at the exit cell?
Path cost	Cost of each move (steps, time or battery used). A turn or a rough floor may cost more than a straight move.

Note: A maze has loops, so the robot must remember cells it has already visited on the current path to avoid going round and round.

3. How the Two Strategies Solve the Maze

3.1 Uniform Cost Search (UCS)

UCS always expands the node with the **lowest total path cost so far** g(n). It keeps the frontier in a **priority queue** ordered by cost.

Steps in the maze:

- Put the start cell in the queue with cost 0.
- Remove the cheapest cell from the queue. If it is the exit, stop.
- Otherwise, add all its neighbour cells with their total cost, and repeat.

Help to the robot: If moves have different costs, UCS finds the **cheapest route** to the exit. If all moves cost the same, UCS works like Breadth-First Search and finds the route with the fewest moves.

3.2 Iterative Deepening Search (IDS)

IDS runs **Depth-Limited Search again and again** with limits 0, 1, 2, 3 … until the exit is found. Each round goes deep like Depth-First Search, but never beyond the current limit.

Steps in the maze:

- Search up to 0 moves, then 1 move, then 2 moves, and so on.
- In every round, go down one path as far as the limit allows, then back up and try the next path.

- Stop in the first round where the exit is found.

Help to the robot: IDS needs very little memory (only the current path), yet it still finds the exit with the fewest moves. This suits a small robot with limited memory.

4. Small Worked Example

Take a maze where S is the start and G is the exit. The cost of each move is shown on the path.

Path	Moves (depth)	Total cost
Path 1: S → A → G (S–A = 2, A–G = 6)	2	8
Path 2: S → B → C → G (S–B = 3, B–C = 2, C–G = 2)	3	7

UCS: It expands S (cost 0), then A (2), then B (3), then C (5), and reaches G through C with cost 7, which is lower than 8. So UCS chooses **Path 2 (cost 7)**.

IDS: With limit 2 it finds G through A. So IDS chooses **Path 1 (2 moves, cost 8)**.

This shows that IDS gives the path with the **fewest moves**, while UCS gives the path with the **lowest cost**. They are the same only when every move costs the same.

5. Advantages and Disadvantages (Time and Space)

Here b = number of choices at each junction (branching factor), d = number of moves to the nearest exit, C* = cost of the best path and ε = smallest cost of one move.

Point	UCS	IDS
Complete?	Yes, if every move cost is greater than 0	Yes, if b is finite
Optimal?	Yes, always gives the lowest-cost path	Yes, only when all moves cost the same
Time complexity	O(b^(1 + C*/ε)) – grows very fast	O(b^d) – grows fast but repeats work
Space complexity	O(b^(1 + C*/ε)) – stores all frontier cells (high)	O(b·d) – stores only the current path (low)
Advantages	• Finds the cheapest path • Works with different move costs	• Very low memory • No need to know the depth in advance • Fewest-move path
Disadvantages	• Needs a lot of memory • Explores in all directions, so it is slow in large mazes	• Repeats work in every round (small extra cost, because most cells lie at the deepest level) • Not optimal when costs differ

6. Relation to Types of Intelligent Agents

Agent type	How it behaves in the maze	Effect on decision-making
Simple reflex	Uses rules like “if wall ahead, turn right”.	Uses no search and no memory. It can walk in circles forever and may never find the exit.

Agent type	How it behaves in the maze	Effect on decision-making
Model-based	Remembers the visited cells, dead ends and the part of the maze it has mapped.	Gives the search algorithm a state space to work on and avoids repeating mistakes.
Goal-based	Its goal is "reach the exit". It uses UCS or IDS to plan a series of moves.	Decides by thinking ahead instead of reacting. Search is how it plans.
Utility-based	Compares paths by length, safety and battery use.	Chooses the best path, not just any path. UCS matches this because it minimises path cost.

How agent design affects decisions: The more an agent can remember, plan and compare, the better its decisions become. A simple reflex agent reacts only to what it sees now, a model-based agent uses memory, a goal-based agent plans with search, and a utility-based agent chooses the best option.

7. Best Combination and Justification

Suggested choice: a Goal-Based Agent with an internal model (map memory) using Iterative Deepening Search (IDS).

Why this is best:

- **Unknown maze:** The robot must build a map of what it has seen, so a model is needed.
- **Clear goal:** The only aim is to reach the exit, so a goal-based agent fits.
- **Low memory:** IDS needs only O(b·d) space, whereas UCS may fill the robot's memory in a big maze.
- **Safe results:** IDS is complete and finds the path with the fewest moves, so the robot will not get stuck.
- **Loop control:** The map memory stops the robot from visiting the same cells again.

Alternative: If different moves have very different costs (for example, slopes or sharp turns that use more battery), use a **utility-based agent with UCS**, because only UCS guarantees the lowest-cost path.

8. Conclusion

UCS finds the **cheapest** path but needs more time and memory. IDS finds the **shortest** path with very little memory but repeats some work. Agent design decides how well these searches are used. For an unknown maze, a **goal-based agent with a model and IDS** is the most effective and practical combination.